

Lab Assignment-19_1

Name:Anand

Hallticket:2303A51090

Batch-02

Task 1 – Python to Java Conversion

Task:

Translate a simple Python class into Java using AI assistance.

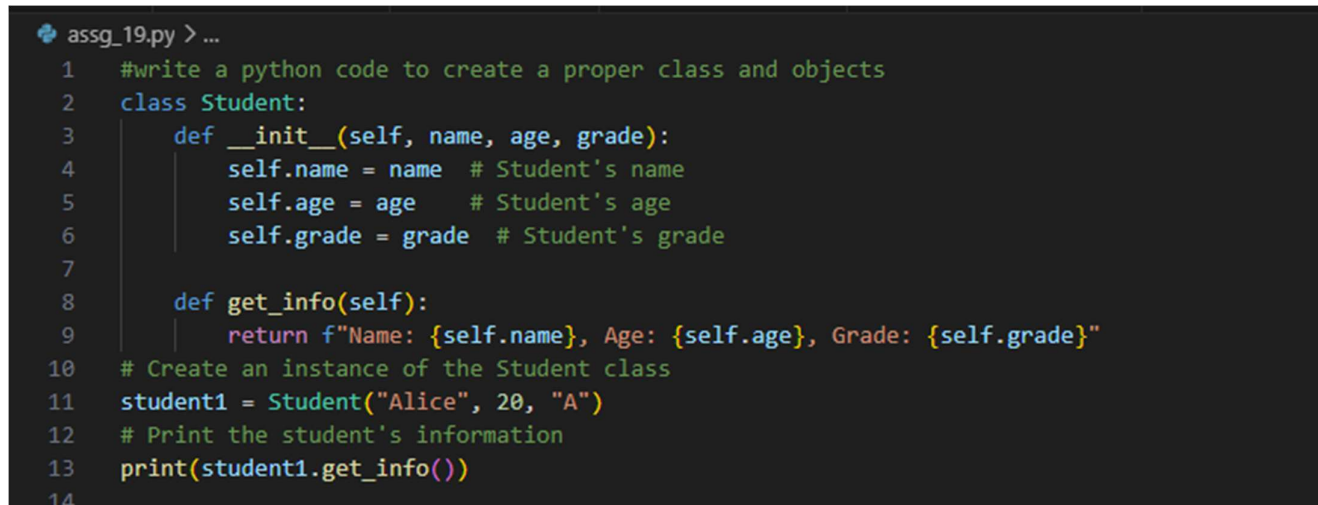
Instructions:

- Prompt AI to convert a Python class with attributes and methods into equivalent Java code.
- Verify that constructors, method signatures, and data types are properly handled.
- Run both versions (Python & Java) to compare expected outputs.

Expected Output:

- Equivalent working Java class translated from Python.

Screenshots:



```
assg_19.py > ...
1  #write a python code to create a proper class and objects
2  class Student:
3      def __init__(self, name, age, grade):
4          self.name = name # Student's name
5          self.age = age    # Student's age
6          self.grade = grade # Student's grade
7
8      def get_info(self):
9          return f"Name: {self.name}, Age: {self.age}, Grade: {self.grade}"
10 # Create an instance of the Student class
11 student1 = Student("Alice", 20, "A")
12 # Print the student's information
13 print(student1.get_info())
14
```

```

14
15 #convert the above into java code
16 public class Student {
17     private String name; // Student's name
18     private int age; // Student's age
19     private String grade; // Student's grade
20
21     // Constructor to initialize the student's attributes
22     public Student(String name, int age, String grade) {
23         this.name = name;
24         this.age = age;
25         this.grade = grade;
26     }
27
28     // Method to get the student's information
29     public String getInfo() {
30         return "Name: " + name + ", Age: " + age + ", Grade: " + grade;
31     }
32
33     // Main method to create an instance of Student and print info
34     public static void main(String[] args) {
35         Student student1 = new Student("Alice", 20, "A");
36         System.out.println(student1.getInfo());
37     }
38 }
39

```

Output:

```
Name: Alice, Age: 20, Grade: A
```

Task 2 – C++ to Python Function Conversion

Task:

Convert a C++ function for factorial calculation into Python.

Instructions:

- Ask AI to translate a recursive C++ factorial function into Python.
- Validate correctness by testing with sample inputs.
- Ensure Python version follows Pythonic conventions.

Expected Output:

- A correct Python implementation equivalent to the original C++ factorial function.

Screenshots:

```

41  #write a c++ code function that calculates the factorial of a number
42  #include <iostream>
43  using namespace std;
44  // Function to calculate factorial
45  unsigned long long factorial(int n) {
46      if (n < 0) {
47          throw invalid_argument("Factorial is not defined for negative numbers.");
48      }
49      unsigned long long result = 1;
50      for (int i = 1; i <= n; ++i) {
51          result *= i;
52      }
53      return result;
54  }
55  int main() {
56      int number;
57      cout << "Enter a number to calculate its factorial: ";
58      cin >> number;
59      try {
60          unsigned long long fact = factorial(number);
61          cout << "Factorial of " << number << " is " << fact << endl;
62      } catch (const invalid_argument& e) {
63          cout << "Error: " << e.what() << endl;
64      }
65      return 0;
66  }

```

```

67  #write above into python code
68  def factorial(n):
69      if n < 0:
70          raise ValueError("Factorial is not defined for negative numbers.")
71      result = 1
72      for i in range(1, n + 1):
73          result *= i
74      return result
75  try:
76      number = int(input("Enter a number to calculate its factorial: "))
77      fact = factorial(number)
78      print(f"Factorial of {number} is {fact}")
79  except ValueError as e:
80      print(f"Error: {e}")
81

```

Output:

```

Enter a number to calculate its factorial: 10
Factorial of 10 is 3628800
PS C:\Users\arell\Music\aiac>

```

Task 3 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in

Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

Screenshots:

```
82 #write a sql commands that create a table named as employee and insert some data and retrieve the same and salary which are salary greater than 50000
83 -- Create the employee table
84 CREATE TABLE employee (
85     id INT PRIMARY KEY,
86     name VARCHAR(50),
87     position VARCHAR(50),
88     salary DECIMAL(10, 2)
89 );
90 -- Insert some data into the employee table
91 INSERT INTO employee (id, name, position, salary) VALUES
92 (1, 'John Doe', 'Software Engineer', 60000.00),
93 (2, 'Jane Smith', 'Data Analyst', 45000.00),
94 (3, 'Emily Davis', 'Project Manager', 75000.00),
95 (4, 'Michael Brown', 'Intern', 30000.00);
96
97 -- Retrieve employees with salary greater than 50000
98 SELECT * FROM employee WHERE salary > 50000;
99
```

```
100 #write the above sql commands into python using pandas dataframe
101 import pandas as pd
102 # Create a DataFrame to represent the employee table
103 data = {
104     'id': [1, 2, 3, 4],
105     'name': ['John Doe', 'Jane Smith', 'Emily Davis', 'Michael Brown'],
106     'position': ['Software Engineer', 'Data Analyst', 'Project Manager', 'Intern'],
107     'salary': [60000.00, 45000.00, 75000.00, 30000.00]
108 }
109 employee_df = pd.DataFrame(data)
110 # Retrieve employees with salary greater than 50000
111 high_salary_employees = employee_df[employee_df['salary'] > 50000]
112 print(high_salary_employees)
113
```

Output:

```
   id  name      position  salary
0  1  John Doe  Software Engineer  60000.0
2  3  Emily Davis  Project Manager  75000.0
PS C:\Users\arell\Music\aiac>
```

Task 4 – Real-Time Application: Multi-Language Snippet

Converter

Scenario:

Students will choose a real-time use case where they need code in multiple languages. Example:

- Converting a Python web scraping snippet into JavaScript for browser automation.
- Translating a Java method into C# for enterprise applications.

Instructions:

- Prompt AI to translate code between two chosen languages.
- Test both implementations to ensure they produce the same results.
- Document challenges faced during translation (e.g., syntax, libraries).

Expected Output:

- Working equivalent code snippets in at least two different programming languages.

SCREENSHOTS:

```
115 #write a code to calculate the fibonacci series up to n terms
116 def fibonacci(n):
117     if n < 0:
118         raise ValueError("Number of terms must be non-negative.")
119     fib_series = []
120     a, b = 0, 1
121     for _ in range(n):
122         fib_series.append(a)
123         a, b = b, a + b
124     return fib_series
125 try:
126     n_terms = int(input("Enter the number of terms for Fibonacci series: "))
127     print(f"Fibonacci series up to {n_terms} terms: {fibonacci(n_terms)}")
128 except ValueError as e:
129     print(f"Error: {e}")
```

```

130 #WRITE THE ABOVE CODE INTO JAVA
131 import java.util.Scanner;
132 public class Fibonacci {
133     public static void main(String[] args) {
134         Scanner scanner = new Scanner(System.in);
135         System.out.print("Enter the number of terms for Fibonacci series: ");
136         int nTerms = scanner.nextInt();
137         try {
138             System.out.println("Fibonacci series up to " + nTerms + " terms: " + fibonacci(nTerms));
139         } catch (IllegalArgumentException e) {
140             System.out.println("Error: " + e.getMessage());
141         }
142         scanner.close();
143     }
144
145     public static String fibonacci(int n) {
146         if (n < 0) {
147             throw new IllegalArgumentException("Number of terms must be non-negative.");
148         }
149         StringBuilder fibSeries = new StringBuilder();
150         int a = 0, b = 1;
151         for (int i = 0; i < n; i++) {
152             fibSeries.append(a).append(" ");
153             int temp = a;
154             a = b;
155             b = temp + b;
156         }
157         return fibSeries.toString().trim();
158     }
159 }

```

Output:

```

PS C:\Users\arell\Music\aiac> cd "c:\Users\arell\Music\aiac" ; if ($?) { javac Fibonacci.java } ; if ($?) { java Fibonacci }
Enter the number of terms for Fibonacci series: 10
Fibonacci series up to 10 terms: 0 1 1 2 3 5 8 13 21 34
PS C:\Users\arell\Music\aiac> python -u "c:\Users\arell\Music\aiac\assg_19.py"
Enter the number of terms for Fibonacci series: 10
Fibonacci series up to 10 terms: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
PS C:\Users\arell\Music\aiac>

```